System Design & Distributed Scalability Reference Guide

Category: System Architecture & Distributed Systems

Section 1: Horizontal vs Vertical Scaling

Vertical scaling (scaling up) adds more compute, RAM, or faster NVMe disks to an existing single machine. It is simple but bounded by physical hardware limits and introduces a single point of failure (SPOF). Horizontal scaling (scaling out) adds more commodity instances to a distributed cluster, requiring load balancers and stateless application servers.

Section 2: Caching Strategies (Cache-Aside, Write-Through, Write-Behind)

In Cache-Aside (Lazy Loading), the application queries Redis first; on a cache miss, it fetches data from PostgreSQL, populates Redis, and returns the result. In Write-Through, updates write to the cache and the database synchronously, ensuring consistency at the expense of write latency. In Write-Behind (Write-Back), updates write to the cache immediately and asynchronously flush to disk, boosting write throughput.

Section 3: Database Sharding and Consistent Hashing

Sharding partitions large database tables horizontally across distinct database servers using a shard key (e.g., hash(user_id) % num_shards). To prevent catastrophic re-shuffling when nodes are added or removed, distributed architectures employ Consistent Hashing mapped onto a 360-degree virtual ring with virtual nodes (vnodes) to evenly balance shard load.